

01장 준비하기 (Preparation)

준비하기 개요

파이썬 GUI 프로그래밍을 시작하기 전에, 제대로 된 출발선에 서는 것이 중요합니다. 빨리 결과물을 만들고 싶은 마음에 준비 단계를 서두르다 보면, 이후 예상치 못한 오류나 불편함으로 돌아오는 경우가 많습니다. 처음 환경을 갖추는 과정이 낯설고 번거롭게 느껴질 수 있지만, 한 번만 제대로 해두면 이후 학습이 훨씬 수월해집니다.

개발 환경이란 코드를 작성하고 실행하며 오류를 수정하는 모든 과정이 편리하게 이루어질 수 있도록 나만의 작업 공간을 만드는 일입니다. 잘 갖추어진 환경은 학습 효율을 높여줄 뿐만 아니라, 문제가 생겼을 때 원인을 찾는 시간도 크게 줄여줍니다.

이 책은 파이썬 기초를 이미 습득한 독자를 대상으로 하지만, 실습 중 필요할 때 바로 찾아볼 수 있도록 핵심 기초 문법을 간략히 정리해 두었습니다.

이 장을 마치고 나면 본격적인 GUI 프로그래밍을 시작할 준비가 완전히 갖추질 것입니다.

목차

- [01-01 파이썬 설치와 가상환경](#)
- [01-02 VS Code 설치](#)
- [01-03 파이썬 문법 요약](#)

파이썬 설치와 가상환경

먼저 파이썬 코드를 작성하고 실행하기 위해 윈도우 PC에서 파이썬을 설치하고 가상환경을 설정하는 방법을 알아보겠습니다.

파이썬 설치를 마치면 프로젝트에 따라 다양한 라이브러리를 선택적으로 사용할 수 있는데, 이를 지원하기 위해 가상환경을 만드는 방법도 함께 살펴보겠습니다.

- [파이썬 설치](#)
 - [설치파일 다운로드](#)
 - [파이썬 설치](#)
 - [설치 확인하기](#)
 - [작업 폴더 만들기](#)
- [가상환경](#)
 - [가상환경이 필요한 이유](#)
 - [가상환경 만들기](#)
 - [가상환경 활성화하기](#)
- [라이브러리 설치](#)
 - [pip 업그레이드하기](#)
 - [GUI 라이브러리 설치해 보기](#)
- [오프라인 PC에서 pip 사용하기](#)
 - [오프라인 PC 파이썬 버전확인](#)
 - [인터넷 되는 PC에서 패키지 내려받기](#)
 - [오프라인 PC로 파일 옮기기](#)
 - [오프라인 pc에서 설치하기](#)

파이썬 설치

- 윈도우에서 파이썬을 설치하는 방법은 크게 두 가지입니다.
 - 첫 번째는 python.org 공식 웹사이트 설치 파일을 사용하는 방법입니다.
 - 두 번째는 Microsoft Store 또는 Python Install Manager를 사용하는 방법입니다.

- 라이브러리 관리와 경로 설정의 자유도를 위해 python.org 공식 웹사이트에서 .exe 설치파일을 이용한 설치를 권장합니다.

설치파일 다운로드

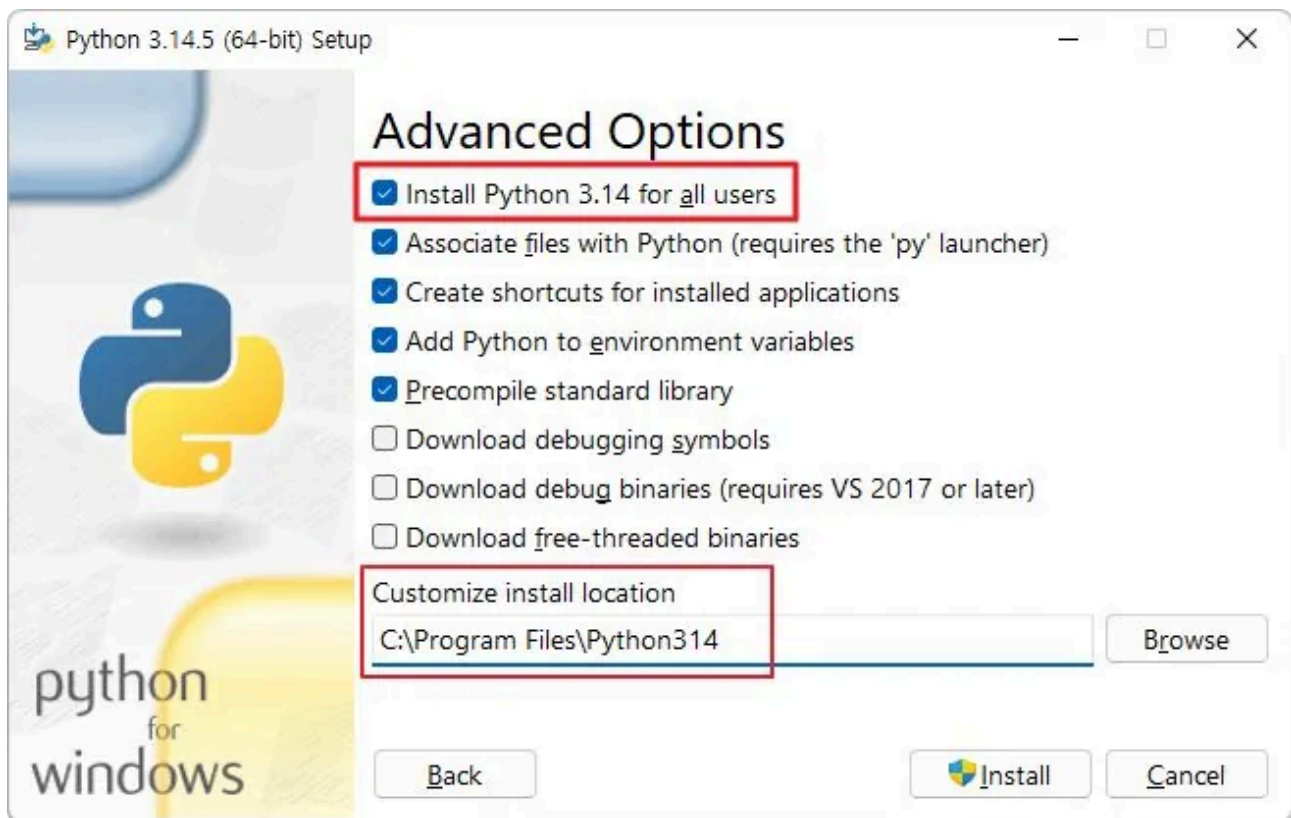
- <https://www.python.org/> 공식 웹사이트에서 'Downloads > Windows' 클릭합니다.
- Stable Releases 버전 중 가장 최신버전의 'Windows installer (64-bit)' 를 클릭해 다운로드 합니다.
(2026.05.11. 기준 Python 3.14.5)

파이썬 설치

- 다운로드 받은 설치 파일을 실행합니다.
- 설치 화면에서 반드시 'Add python.exe to PATH' 항목을 체크합니다.



- Advanced Options 화면에서 'Install Python for all users' 체크하고 하단 'Customize install location'을 그림과 같이 설정합니다.



설치 확인하기

- 설치가 끝났다면 PowerShell 또는 명령 프롬프트를 실행합니다.
- 다음 명령을 입력합니다.

```
python --version
```

- 정상적으로 설치되었다면 다음과 비슷한 결과가 출력됩니다.

```
Python 3.14.5
```

작업 폴더 만들기

- GUI 프로그램은 하나의 파일로 시작할 수 있지만, 학습이 진행될수록 이미지, 설정 파일, 데이터 파일, 여러 개의 파이썬 파일을 함께 사용하게 됩니다.
- 먼저 실습용 폴더(D:\python-gui)를 만들어 보겠습니다.

```
mkdir D:\python-gui
cd D:\python-gui
```

위 명령은 D 드라이브에 python-gui 폴더를 만들고 그 폴더로 이동합니다.

가상환경

가상환경이 필요한 이유

- 파이썬 가상환경은 프로젝트별로 독립된 실행 환경을 구축하여 패키지 간의 간섭을 차단합니다.
- 전역(Global) 환경에 모든 라이브러리를 설치할 경우 발생하는 의존성 충돌 문제를 해결하고 개발 환경의 일관성을 유지하기 위해 사용합니다.

가상환경 만들기

- 파이썬에는 venv라는 가상환경 생성 도구가 기본으로 포함되어 있습니다. 별도의 프로그램을 설치하지 않아도 사용할 수 있습니다.
- 현재 python-gui 폴더 안에서 다음 명령을 실행합니다.

```
python -m venv .venv
```

여기서 .venv는 가상환경 폴더 이름입니다.

- 명령 실행 후 프로젝트 폴더 안에는 다음과 같은 구조가 생깁니다.

```
python-gui
├─ .venv
│   ├── Include
│   ├── Lib
│   ├── Scripts
│   └─ pyvenv.cfg
```

- 가상환경에는 독립된 파이썬 실행 파일과 패키지 설치 공간이 만들어집니다.

가상환경 활성화하기

- 가상환경을 만들고 활성화해야 사용할 수 있습니다. 명령 프롬프트에서는 다음 명령을 사용합니다.

```
.venv\Scripts\activate.bat
```

- 정상적으로 활성화되면 프롬프트 앞에 (.venv)가 표시됩니다.

```
(.venv) D:\python-gui>
```

- 파이썬 실행 파일의 위치를 확인합니다.

```
where python
```

결과에 .venv\Scripts\python.exe가 포함되어 있다면 가상환경이 제대로 적용된 것입니다.

- 작업이 끝나고 가상환경에서 빠져나오려면 다음 명령을 실행합니다.

```
deactivate
```

라이브러리 설치

pip 업그레이드하기

- pip**는 파이썬 패키지를 설치하는 도구입니다. 가상환경을 활성화한 상태에서 다음 명령을 실행합니다.

```
python -m pip install --upgrade pip
```

GUI 라이브러리 설치해 보기

- 이 책에서는 GUI 라이브러리로 PySide6 를 사용합니다. 가상환경이 활성화된 상태에서 다음 명령을 실행합니다.

```
python -m pip install Pyside6
```

- 설치가 완료되면 다음 명령으로 정상적으로 설치되었는지 확인합니다.

```
python -m pip list
```

오프라인 PC에서 pip 사용하기

오프라인 PC 파이썬 버전확인

- 먼저 오프라인 PC의 파이썬 버전과 플랫폼을 CMD창에서 확인합니다.

```
python --version
python -c "import platform; print(platform.machine())"
```

- 다음과 비슷한 값이 출력됩니다.

```
Python 3.13.14
AMD64
```

인터넷 되는 PC에서 패키지 내려받기

- `pip download` 명령을 사용하면 패키지를 설치하지 않고 파일 (`.whl` , `.tar.gz`)만 지정한 디렉터리에 내려 받습니다.
- 앞 서 확인한 파이썬 버전과 플랫폼에 맞는 라이브러리를 다운로드 받습니다.

```
python -m pip download requests -d D:\offline-packages --python-version 3.13 --platform wi
```


오프라인 PC로 파일 옮기기

- D:\offline-packages 폴더 전체를 USB 등에 복사해 오프라인 PC로 옮깁니다.

오프라인 pc에서 설치하기

- 오프라인 PC에서 가상환경을 활성화한 뒤, --no-index와 --find-links 옵션으로 인터넷 대신 옮겨 온 폴더에서 패키지를 찾아 설치합니다.

```
python -m pip install --no-index --find-links=D:\offline-packages requests
```

--no-index 는 PyPI 접속을 하지 않도록 하고, --find-links 는 지정한 폴더에서 패키지 파일을 찾도록 합니다

VS Code 설치

앞 장에서 파이썬 설치와 가상환경 설정을 마쳤습니다. 이제 본격적으로 코드를 작성하고 실행할 편집기(Editor)가 필요합니다.

파이썬 코드는 메모장으로도 작성할 수 있지만, 자동 완성, 디버깅, 가상환경 연동 같은 기능을 갖춘 편집기를 사용하면 학습과 개발 효율이 크게 올라갑니다. 이 책에서는 파이썬 개발에 두루 활용되는 Visual Studio Code (VS Code)를 사용합니다.

- [VS Code란?](#)
- [VS Code 설치](#)
 - [설치파일 다운로드](#)
 - [VS Code 설치](#)
- [Python 확장 설치](#)
 - [설치 방법](#)
 - [함께 설치되는 확장](#)
- [작업 폴더와 인터프리터 선택](#)
 - [작업 폴더 열기](#)
 - [파이썬 인터프리터 선택](#)
- [첫 번째 파이썬 파일 실행하기](#)
 - [파일 만들기](#)
 - [코드 작성](#)
 - [실행하기](#)

VS Code란?

- VS Code는 마이크로소프트에서 만든 무료 코드 편집기로, 윈도우, macOS, 리눅스에서 모두 동작합니다.
- 가벼우면서도 확장(Extension)을 통해 다양한 언어와 기능을 추가할 수 있습니다.
- 파이썬 공식 확장이 잘 갖춰져 있어, 가상환경 인식·자동 완성·디버깅을 별도 설정 없이 사용할 수 있습니다.

VS Code 설치

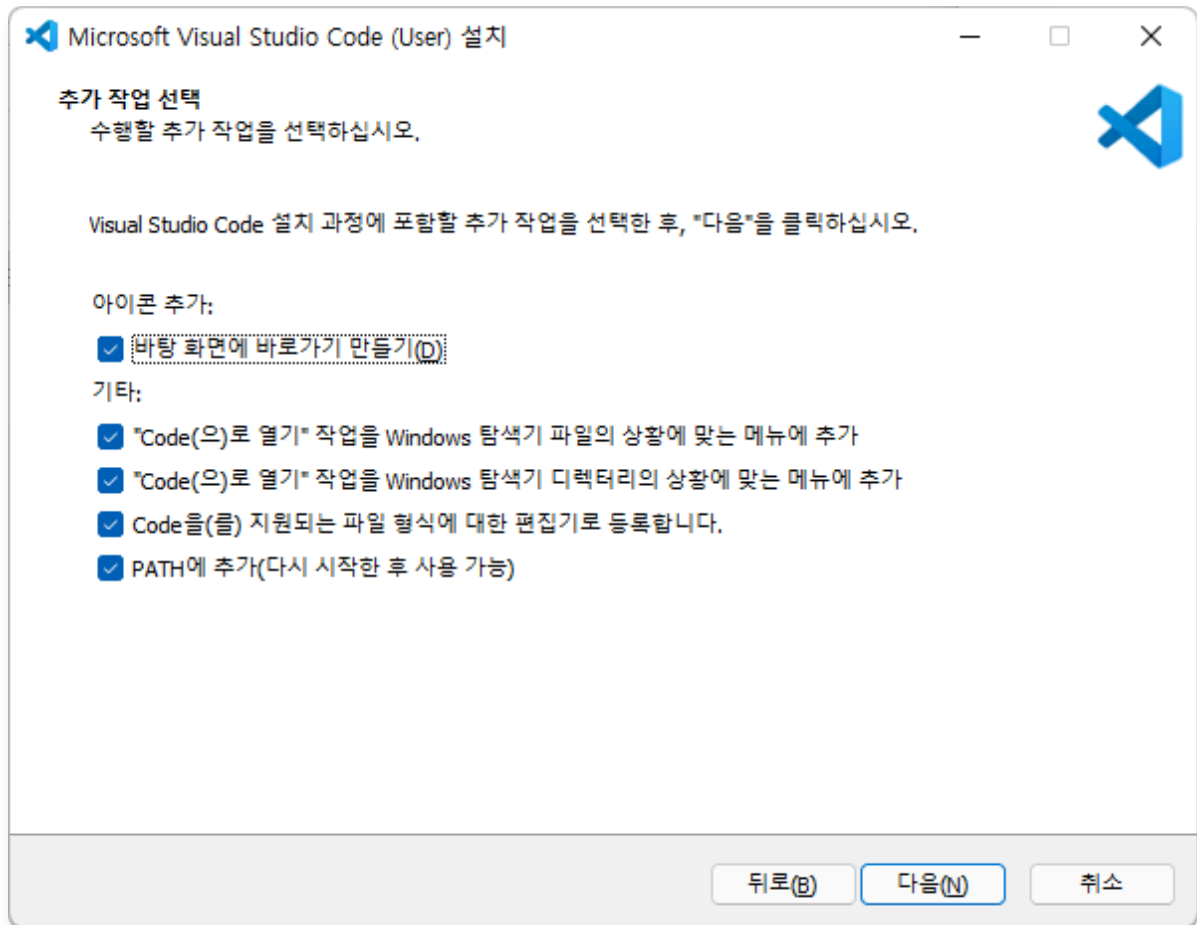
설치파일 다운로드

- <https://code.visualstudio.com> 공식 웹사이트에 접속합니다.
- 메인 화면의 'Download for Windows' 버튼을 클릭해 설치 파일(.exe)을 다운로드합니다.

VS Code 설치

- 다운로드 받은 설치 파일을 실행합니다.
- 사용권 계약 화면에서 '동의함' 을 선택하고 진행합니다.

- 추가 작업 선택 화면에서 다음 항목을 모두 체크하는 것을 권장합니다.
 - 바탕 화면에 바로 가기 만들기
 - "Code(으)로 열기" 작업을 Windows 탐색기 파일의 상황에 맞는 메뉴에 추가
 - "Code(으)로 열기" 작업을 Windows 탐색기 디렉터리의 상황에 맞는 메뉴에 추가
 - Code을(를) 지원되는 파일 형식에 대한 편집기로 등록합니다.
 - PATH에 추가(다시 시작한 후 사용 가능) ← 반드시 체크



'PATH에 추가' 옵션은 명령 프롬프트나 PowerShell에서 `code` 명령으로 VS Code를 실행할 수 있게 해줍니다. 꼭 체크하세요.

- '설치' 버튼을 클릭하면 설치가 진행됩니다.
- 설치가 끝나면 'Visual Studio Code 실행' 항목을 체크하고 '종료'를 누릅니다.

Python 확장 설치

- VS Code 자체에는 파이썬을 실행하는 기능이 들어있지 않습니다.
- Microsoft에서 제공하는 Python 확장을 설치해야 파이썬 파일을 인식하고, 자동 완성·디버깅·가상환경 연동 같은 기능을 사용할 수 있습니다.

설치 방법

- 왼쪽 사이드바의 확장 아이콘을 클릭합니다. (Ctrl + Shift + X)
- 검색창에 Python 을 입력합니다.
- 검색 결과 가장 위에 표시되는 Python 확장 (게시자: Microsoft) 을 선택하고 설치 버튼을 클릭합니다.



함께 설치되는 확장

- Python 확장을 설치하면 다음 확장이 자동으로 함께 설치됩니다. 별도로 설치할 필요는 없습니다.
 - Pylance : 자동 완성, 타입 검사, 코드 탐색 기능 제공
 - Python Debugger : 파이썬 코드 디버깅 지원
 - Python Environments : 가상환경과 패키지를 통합 관리

작업 폴더와 인터프리터 선택

작업 폴더 열기

- 앞 장에서 만든 D:\python-gui 폴더를 VS Code에서 열어보겠습니다.
- VS Code 메뉴에서 파일 > 폴더 열기 를 선택합니다. (Ctrl + K, Ctrl + O)
- 파일 탐색기에서 D:\python-gui 폴더를 선택하고 폴더 선택 을 클릭합니다.

- 처음 폴더를 열면 "이 폴더에 있는 파일의 작성자를 신뢰합니까?" 라는 안내가 나타납니다. 본인 이 만든 폴더이므로 예, 작성자를 신뢰합니다 를 선택합니다.

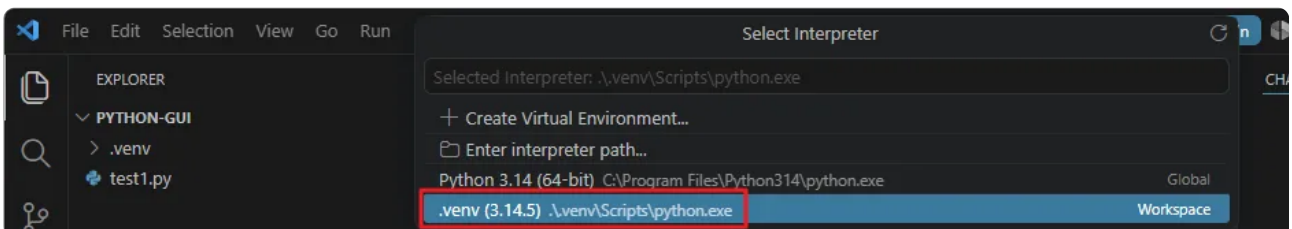
파일 탐색기에서 D:\python-gui 폴더를 마우스 오른쪽 클릭한 뒤 Code(으)로 열기를 선택 해도 됩니다. (설치 시 해당 옵션을 체크한 경우)

- 왼쪽 사이드바의 탐색기(Explorer) 패널에 PYTHON-GUI 폴더와 .venv 하위 폴더가 표시되면 정상 입니다.

파이썬 인터프리터 선택

- VS Code는 시스템에 설치된 여러 파이썬을 탐지할 수 있습니다. 앞 장에서 만든 가상환경(.venv) 을 사용하도록 지정해 보겠습니다.
 - Ctrl + Shift + P 를 눌러 명령 팔레트(Command Palette) 를 엽니다.
 - 검색창에 Python: Select Interpreter 를 입력하고 선택합니다.
 - 인터프리터 목록이 나타납니다. 다음과 같이 표시되는 항목을 선택합니다.

```
Python 3.14.5 ('.venv': venv) .\venv\Scripts\python.exe
```



- 선택이 끝나면 VS Code 창 오른쪽 아래에 현재 인터프리터가 표시됩니다.

```
3.14.5 ('.venv': venv)
```

첫 번째 파이썬 파일 실행하기

- 이제 실제로 파이썬 코드를 작성하고 실행해 보겠습니다.

파일 만들기

- 탐색기 패널에서 PYTHON-GUI 옆의 새 파일 아이콘을 클릭합니다.
- 파일 이름을 hello.py 로 입력하고 엔터를 누릅니다.

코드 작성

- hello.py 파일에 다음 코드를 입력합니다.

```
print("Hello, Python!")

name = "VS Code"
print(f"안녕하세요, {name}에서 파이썬을 실행합니다.")
```

실행하기

- 방법 1. 실행 버튼 사용
 - 편집기 우측 상단의 ► (Run Python File) 버튼을 클릭합니다.
 - 화면 아래쪽 터미널에 코드 실행 결과가 출력됩니다.
- 방법 2. 단축키 사용
 - 편집기에 포커스를 둔 상태로 Ctrl + F5 를 누르면 디버깅 없이 실행됩니다.
- 방법 3. 터미널에서 직접 실행
 - Ctrl + (백틱) 을 눌러 통합 터미널 을 엽니다.
 - 프롬프트 앞에 (.venv) 표시가 있는지 확인합니다. (가상환경이 자동 활성화됩니다.)
 - 다음 명령을 입력합니다.

```
python hello.py
```

알아두기

앞선 강좌에서 `python -m pip install PySide6` 명령어로 PySide6를 설치했다면, Qt Designer도 함께 설치됩니다.

VS Code 터미널 프롬프트에서 `pyside6-designer` 명령어를 입력하면 Qt Designer를 실행할 수 있습니다.

파이썬 핵심 요약

이 책은 파이썬 문법을 이미 익힌 독자를 대상으로 합니다.

따라서 이 장은 문법을 처음 가르치려는 것이 아니라, 본격적인 GUI 프로그래밍에 앞서 핵심 문법을 빠르게 되짚어 볼 수 있도록 짧은 예제 중심으로 간결하게 정리했습니다. 필요할 때 참고 자료로 활용하시기 바랍니다.

- [파이썬 기본 문법](#)
 - [기본규칙](#)
 - [주석](#)
 - [변수와 자료형](#)
- [연산자](#)
 - [산술 연산자](#)
 - [비교/논리 연산자](#)
- [제어문](#)
 - [조건문](#)
 - [삼항 연산자](#)
 - [반복문](#)
- [자료구조](#)
 - [리스트](#)
 - [딕셔너리](#)
 - [튜플 & 세트](#)
- [함수, 파일 입출력](#)
 - [함수](#)
 - [람다\(lambda\) 함수](#)
 - [값 입력 받기](#)
 - [파일 입출력](#)
- [예외 처리](#)

- [객체지향\(OOP\)](#)
 - [클래스와 객체](#)
 - [생성자와 메서드](#)
 - [상속](#)
- [모듈과 라이브러리](#)

파이썬 기본 문법

기본규칙

- 파이썬은 중괄호 `{ }` 대신 **들여쓰기**로 코드 블록을 구분합니다.
- 보통 들여쓰기는 **스페이스 4칸**을 사용합니다.

주석

- 주석은 코드에 설명을 적는 용도이며, 실행에는 영향을 주지 않습니다.
- 한 줄 주석은 `#` 뒤에 작성합니다.
- 여러 줄 설명은 `""" """` 또는 `''' '''` 형태를 자주 사용합니다.
 - 엄밀히 말하면 이것은 **주석이 아니라 여러 줄 문자열**입니다.
 - 함수나 클래스 설명(docstring)으로 많이 사용합니다.

변수와 자료형

- 파이썬은 변수의 자료형 미리 선언하지 않고 값에 따라 자동 결정됩니다.
- `type()`으로 자료형 확인 가능하며 `int()`, `float()`, `str()` 등으로 형 변환할 수 있습니다.

```

# 변수 선언과 자료형
x = 10                # 정수(int)
y = 3.14              # 실수(float)
name = "Tom"          # 문자열(str)
flag = True           # 불리언(bool)

print(type(x))        # <class 'int'>
print(type(y))        # <class 'float'>
print(type(name))     # <class 'str'>
print(type(flag))     # <class 'bool'>

# 형 변환
print(int(3.7))       # 3
print(float(5))       # 5.0
print(str(100))       # 100

```

- 문자열(String)은 작은따옴표(')나 큰따옴표(")로 묶어 표현합니다.

```

s = "It's never too late to learn. "

print(len(s))         # 30
print(s[0:4])         # "It's"
print(s.replace("never", "")) # It's too late to learn.
print(s.find("learn")) # 23
print("late" in s)     # True
print(s.upper())       # 대문자 변환
print(s.rstrip())      # 오른쪽 공백 지우기
print(s.split())       # 공백을 기준으로 문자열 나누어 리스트에 저장

```

연산자

산술 연산자

- 숫자 계산에 사용하는 연산자입니다.

```

a, b = 7, 3
print(a + b)      # 덧셈
print(a - b)      # 뺄셈
print(a * b)      # 곱셈
print(a / b)      # 나눗셈 (실수)
print(a // b)     # 몫
print(a % b)      # 나머지
print(a ** b)     # 거듭제곱

```

비교/논리 연산자

- 비교 연산자 (==, !=, <, >)

```

a, b = 5, 3
print(a == b)    # False
print(a != b)    # True
print(a > b)     # True
print(a < b)     # False
print(a >= 5)    # True
print(b <= 2)    # False

```

- 논리 연산자 (and, or, not)

```

x = True
y = False
print(x and y)   # False : 둘 다 True여야 True
print(x or y)    # True  : 하나라도 True면 True
print(not x)     # False : 반대 값

```

제어문

조건문

- 조건에 따라 다른 코드를 실행할 때 사용합니다.
- if-elif-else 문

```

score = 75

if score >= 90:
    print("A")
elif score >= 70:
    print("B")
else:
    print("C")

# 출력 : B

```

삼항 연산자

- 조건문을 한 줄로 작성해 변수에 값을 할당하는 문법입니다.
- `[참일 때 값] if [조건] else [거짓일 때 값]` 형태를 갖습니다.

```

score = 85
result = "합격" if score > 79 else "불합격"
print(result)
# 출력: 합격

```

반복문

- 정해진 횟수만큼 반복할 때 주로 사용합니다.
- for문

```

for i in range(5):
    print(i, end=" ")

# 출력 : 0 1 2 3 4

```

- 반복문 제어 (break, continue)

```

for i in range(6):      # 0, 1, 2, 3, 4,5 반복
    if i == 1:
        continue      # 1일 때는 건너뛴 (출력 안 함)
    if i == 4:
        break          # 4일 때는 반복문 자체를 완전히 종료함
    print(i, end=" ")

# 출력: 0 2 3

```

- while문

```

count = 0
while count < 3:      # 조건 반복
    print(count, end=" ")
    count += 1

# 출력 : 0 1 2

```

자료구조

리스트

- 리스트는 여러 값을 순서대로 저장할 수 있는 데이터 구조입니다.
- 값이 들어가는 순서가 있고, 번호(인덱스)로 꺼낼 수 있습니다.
- 리스트는 대괄호 []로 표현되며, 각 요소는 쉼표로 구분됩니다.

```

nums = [1, 2, 3, 4]
nums.append(5)      # 요소 추가
nums.remove(3)      # 값 '3'을 찾아 제거
print(nums[0])      # 첫 번째 요소
print(nums[-1])     # 마지막 요소
print(len(nums))    # 길이

```

딕셔너리

- 단어장 처럼 "키(key)"와 "값(value)"을 짝으로 저장하는 구조입니다.

```
person = {"name": "Tom", "age": 25}
print(person["name"])      # Tom
person["age"] = 30         # 값 수정
person["city"] = "Seoul"   # 새 키 추가
for key, value in person.items(): # for 문으로 키와 값을 가져오기
    print(f"{key} : {value}")
```

튜플 & 세트

- 튜플(Tuple)은 리스트와 비슷하지만 수정 불가능합니다.

```
# 튜플 (수정 불가, 순서 있음)
t = (1, 2, 3, 3)
print(t[0])          # 1
print(t.count(3))     # 2 (요소 개수 세기)
print(t.index(2))     # 1 (값 '2'가 있는 인덱스 위치 반환)
```

- 세트(Set)는 중복을 허용하지 않고 집합 연산에 유용합니다.

```
# 세트 (중복 제거, 순서 없음)
s = {1, 2, 2, 3}
print(s)              # {1, 2, 3}
s.add(4)              # 요소 추가
s.remove(2)           # 요소 제거
print(3 in s)         # True (포함 여부 확인)

# 집합 연산
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)          # 합집합 {1, 2, 3, 4, 5}
print(a & b)           # 교집합 {3}
print(a - b)          # 차집합 {1, 2}
```

함수, 파일 입출력

함수

- 함수는 특정 작업을 묶어 재사용할 수 있는 코드 블록으로 '재사용성', '가독성', '유지보수 용이' 등의 이점이 있습니다.

```
def add(a, b):
    return a + b

print(add(7, 5))

# 출력 : 12
```

더하기 함수를 정의해 계산기 처럼 사용할 수 있습니다.

```
def is_even(number):
    if number % 2 == 0:
        return True
    else:
        return False

print(is_even(4)) # True
print(is_even(7)) # False
```

조건문과 함께 함수가 "판단" 역할도 할 수 있습니다.

람다(lambda) 함수

- 이름 없이 한 줄로 간단하게 정의하는 함수입니다.
- **lambda 매개변수: 반환값** 형태로 작성하며, 간단한 함수를 짧게 표현할 때 유용합니다.


```
# 일반 함수
def add(a, b):
    return a + b

# 같은 기능을 람다로 표현
add = lambda a, b: a + b
print(add(7, 5))    # 12
```

def로 정의한 함수와 동일하게 동작하지만 더 간결합니다.

- map(), filter(), sorted() 등과 함께 자주 사용됩니다.

```
words = ["banana", "kiwi", "apple"]
result = sorted(words, key=lambda w: len(w))
print(result)    # ['kiwi', 'apple', 'banana']
```

값 입력 받기

- input() 함수를 사용해 사용자 값을 입력 받을 수 있습니다.

```
name = input("이름을 입력하세요: ")
print("안녕하세요, ", name, "님!")
```

파일 입출력

- 키보드 입력과 모니터 출력이 아닌 파일을 사용한 입출력을 할 수 있습니다.

```
# 쓰기
with open("data.txt", "w", encoding="utf-8") as f:
    f.write("Hello File\n")

# 읽기
with open("data.txt", "r") as f:
    content = f.read()
    print(content)
```

예외 처리

- 예외(Exception)란? 프로그램 실행 중 발생하는 "오류 상황"을 말합니다.
- 예외 처리를 통해 프로그램이 멈추지 않고 계속 실행되는 "안정성"을 확보할 수 있습니다.
- else와 finally 활용
 - else: 예외가 발생하지 않았을 때 실행할 코드
 - finally: 예외 여부와 관계없이 반드시 실행할 코드

```
try:
    num = int(input("숫자를 입력하세요: "))
except ValueError:
    print("잘못된 입력입니다.")
else:
    print("정상 입력 →", num)
finally:
    print("프로그램을 종료합니다.")
```

- 모든 예외 한꺼번에 처리

```
try:
    num = int(input("숫자를 입력하세요: "))
    print(10 / num)
except Exception as e:
    print("예외 발생:", e)
```

Exception as e 를 쓰면 어떤 에러든 잡아서 메시지를 출력하고 오류 내용을 직접 확인 가능합니다.

객체지향(OOP)

클래스와 객체

- 클래스(Class)는 무언가를 만들기 위한 '설계도'나 '틀'입니다. (예: 봉어빵 틀)
- 객체(Object)는 클래스를 통해 만들어진 실제 '결과물'입니다. (예: 팔 봉어빵, 슈크림 봉어빵)
- 똑같은 틀(클래스)로 성질이 조금씩 다른 여러 개의 객체를 쉽게 만들어낼 수 있습니다.

```
class Dog:
    pass          # 아무 기능이 없는 빈 클래스 생성

dog1 = Dog()     # 첫 번째 객체 생성
dog2 = Dog()     # 두 번째 객체 생성

print(type(dog1)) # <class '__main__.Dog'>
```

생성자와 메서드

- 생성자(**init**): 객체가 만들어질 때 자동으로 실행되며, 객체의 초기 상태(변수)를 설정합니다.
- 메서드(Method): 클래스 안에 정의된 함수로, 객체의 '행동'이나 '기능'을 나타냅니다.
- 클래스 내부의 메서드는 항상 첫 번째 매개변수로 self를 가져야 합니다. self는 만들어진 객체 자기 자신을 의미합니다.

```

class Car:
    # 생성자: 차가 만들어질 때 모델명과 색상을 지정
    def __init__(self, model, color):
        self.model = model    # 속성 (변수)
        self.color = color

    # 메서드: 차의 행동
    def drive(self):
        print(f"{self.color} {self.model}가 달립니다!")

# 객체 생성 (실제 자동차 만들기)
car1 = Car("소나타", "하얀색")
car2 = Car("포르쉐", "빨간색")

# 속성 확인 및 메서드 사용
print(car1.model)    # 소나타
car2.drive()         # 빨간색 포르쉐가 달립니다!

```

상속

- 기존 클래스의 기능(속성과 메서드)을 그대로 물려받아 새로운 클래스를 만드는 기능입니다.
- 코드를 처음부터 다시 짤 필요 없이, 기존 코드를 재사용하고 필요한 기능만 추가할 때 매우 유용합니다.

```

# 부모 클래스
class Animal:
    def speak(self):
        print("소리를 냅니다.")

# 자식 클래스 (Animal을 상속받아 괄호 안에 적어줌)
class Cat(Animal):
    def meow(self):
        print("야옹!")

cat = Cat()
cat.speak()    # 부모에게서 물려받은 메서드 사용 -> 소리를 냅니다.
cat.meow()     # 자식 클래스에서 새로 만든 메서드 사용 -> 야옹!

```

모듈과 라이브러리

- 모듈은 파이썬 코드가 담긴 파일(.py) 입니다.
- 라이브러리는 여러 모듈의 모음입니다.
- import를 사용해 불러오고, 모듈명.함수()로 사용합니다.

```
import math  
print(math.sqrt(16)) # 제곱근
```